Lifelong Learning for a Kresling-Origami Robot:

A Plug-and-play Module for Supervised Learning of Target Adaptations

E. De Vroey

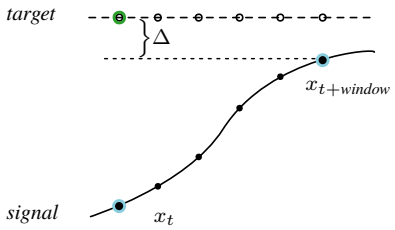
***Abstract*—…**

The method proposed in this paper aims to provide a way of improving a given controller's performance as well as fortifying it against evolving system dynamics *without* having access to the inner workings of the controller itself. In other words, as long as the fundamental assumptions of a controller are satisfied—i.e. it requires a state and a target as input and yields a control action—the method can remain completely agnostic to the specific controller being used. In addition, the aim is not to replace the controller entirely but to introduce an additional module to the control scheme that enables lifelong robustness to evolving system dynamics. The advantage of such a method is that it could be applied to any system as a *plug-and-play* module.

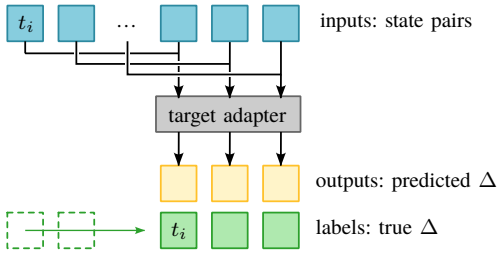
The manner in which this will be achieved is by *target adaptation*. An *adapter* module modifies the target used by the controller to improve its performance. Consider a scenario where a controller yields a control action u to reach a certain desired target state $\mathbf{x}_d$. As the system dynamics evolve, suppose this action u is now insufficient to reach the target. In response, the adapter module *lies* to the controller about reaching $\mathbf{x}_d$ and instead provides it with $\mathbf{x}_d + \Delta$, for which the controller yields a larger action u^* that *does* result in the system reaching state $\mathbf{x}_d$.

The *adapter* module is implemented as a feedforward neural network with to hidden layers with non-linear activations. To train this model to learn suitable target adaptations, interactions of the controller with the system are recorded. The model is then provided with pairs of system states, spaced by a certain time window $\mathbf{x}_t$ and $\mathbf{x}_{t+\text{window}}$, that are obtained from these recordings. From these pairs, the model infers the target it 'believes' the controller aims to reach. However, instead of predicting the target directly, the model learns the difference $\mathbf{\Delta}$ between the target and the latter state in the pair (figs. 1a and 1b).

The model can be trained in a supervised manner (fig. 1b) and then deployed. During deployment, it is presented with the current state $\mathbf{x}_t$ but cannot be presented with $\mathbf{x}_{t+window}$ (since this state is not known yet). Instead, the second state in the pair is set to the desired target. To the model, this represents a situation in which the desired target was reached from the current state within the specified time window. The idea is now that the trained model will produce a $\mathbf{\Delta}$ that, when added to the latter state in the pair (i.e., the desired target), results in an adapted target (fig. 2a) for which the controller will produce a control action. In short, given a current state $\mathbf{x}_t$ and a future state $\mathbf{x}_{t+window}$, the model learned to produce the desired target. Thus, when presented with $\mathbf{x}_t$ and $\mathbf{x}_{t+window} = \mathbf{x}_d$ (supposedly reached within the time window) it provides the (adapted) target that would "push" the controller toward that result. A full overview is presented in figs. 2a and 2b.



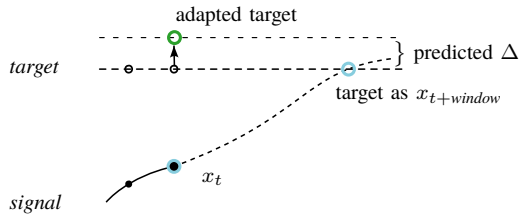
(a) The target adaptation problem: Based on the current position x_t and the position some prediction window later $x_{t+window}$, the model learns to predict the target the controller is trying to reach by representing it as a Δ to be added to $x_{t+window}$.



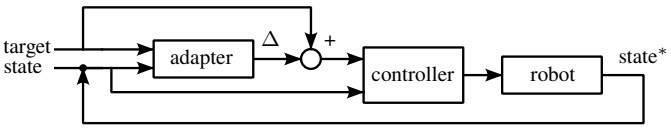
(b) The training process of the target adapter: recorded states spaced by a specified prediction window are paired up and provided as features to the adapter, which learns to predict a $\mathbf{\Delta}$, being the difference between the target $\mathbf{x}_d$ and state $\mathbf{x}_{t+window}$.

Fig. 1: Target adaptation during the training phase

The reason for predicting the difference $\mathbf{\Delta}$ between the state $\mathbf{x}_{t+window}$ and the desired target, rather than a direct prediction of the target, can also be clarified now: By designing the model's output to be the difference $\mathbf{\Delta}$, the model can be initialized with parameters all close to zero (essentially leaving the desired target unchanged) and then steadily improve online as interactions of the controller and system are recorded, without the need for a pretraining phase.



(a) Target adaptation during deployment: By supplying the model with the controller's target position as $x_{t+window}$ it yields a Δ that can be added to the current target, which will supposedly result in the controller reaching the target within the specified window.



(b) Schematic overview of the full implementation: The adapter is used to offset the given target before presenting it to the controller, essentially *lying* to the controller in order to improve its performance.

Fig. 2: Target adaptation during deployment.

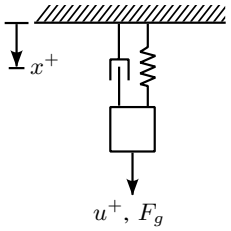


Fig. 3: Setup of the mass-spring-damper system in the toy problem with positive directions indicated.

B. Toy Problem

To explore the proposed method, a toy problem is considered first. The toy problem consists of a simulation of a simple 1D mass-spring-damper system (fig. 3) and an imperfectly tuned PID controller. The mass is suspended in a hanging position with gravity acting on it continuously.

The system state $\mathbf{x}$ consists of the position and velocity x, $\dot{x}$, respectively. The controller aims to reach targets of the form $\mathbf{x}_d = [x_d,\, 0]$, but controls position only. At regular intervals, a random target position x_d is generated for the controller to reach and maintain. Each timestep, the adapter calculates the required Δ, and the controller computes the control action based on the current position and the adapted target position. The system's response to this control input is simulated and the tuple $(x_0, \dot{x}_0, u, x, \dot{x}, x_d + \Delta, 0)$ is added to the buffer. This buffer stores a limited number of tuples corresponding to a specified window of the most recent data. Once capacity is reached, the oldest element is removed at every timestep. At regular update intervals, the adapter is trained on the data in the buffer. It is important that the tuples in the training data always contain the target position that was actually provided to the controller, i.e., $x_d + \Delta$, and not the *true* desired target x_d. As the simulation progresses, the parameters of the dynamic system are gradually altered to simulate system degradation. The pseudo algorithm for the implementation can be found in algorithm 1.

Algorithm 1: Toy problem target adaptation

Data: system state $\mathbf{x}$, target $\mathbf{x}_d$, adaption Δ, control action u
Initialize model: *adapter*;
Initialize buffer: *buffer* $\leftarrow []$;
for *each timestep* t_i **do**
  if *len(buffer) > threshold* **then**
    Remove the oldest element in the buffer;
  end
  if $t_i \mod target\_interval = 0$ **then**
    $\mathbf{x}_d \leftarrow random\_target()$;
  end
  if $t_i \mod update\_interval = 0$ **then**
    (*features, labels*) $\leftarrow$ *prep_data(buffer, window)*;
    adapter.fit(features, labels);
  end
  $\Delta \leftarrow adapter.predict([\mathbf{x}_0, \mathbf{x}_d])$;
  $u \leftarrow controller.compute\_control(\mathbf{x}_0, \mathbf{x}_d + [\Delta, 0])$;
  Simulate the system response $\mathbf{x}$;
  Update system parameters;
  Append $(\mathbf{x}_0, u, \mathbf{x}, \mathbf{x}_d + [\Delta, 0])$ to *buffer*;
  $\mathbf{x}_0 \leftarrow \mathbf{x}$;
end

C. Preliminary Results

The toy problem was implemented in Python: simulating the system response using SciPy's `lsim`, and implementing the adapter in Keras with the PyTorch backend. The simulation ran for 60 seconds at 50 Hz. The target was changed Every 5 seconds, and the adapter was trained fully online, updating every 15 seconds. Training features were created for multiple prediction windows, this means the training features consisted of pairs $[\mathbf{x}_t, \mathbf{x}_{t+window}]$ for windows of 3, 5, 10, 15, and 25. The buffer's budget corresponded to the past 30 seconds of data. For reference, a second controller and system were also simulated, identical to the described simulation, but without the addition of an adapter. To both the reference and adapted signal, a small amount of Gaussian noise was added to simulate measurement noise.

1) A case where the controller is too aggressive: For a first simulation, a combination of a system and controller was chosen where the controller's gains were tuned too high. In fig. 4, the result of the simulation can be observed. At the start, and due to the near-zero initialization of the adapter's parameters, the response signal of the system controlled by the adapted controller is indistinguishable from that of the reference controller. However, as soon as the adapter has trained on the buffer, the changes to the target become visible.

The effects of the adaptation can be more clearly observed in fig. 5. Since the reference controller is too aggressive, the reference signal overshoots and oscillates significantly before settling. In contrast, the adapter counters this by bringing the target closer to the current state. This is the logical result of the buffer containing data corresponding to the overshoots,

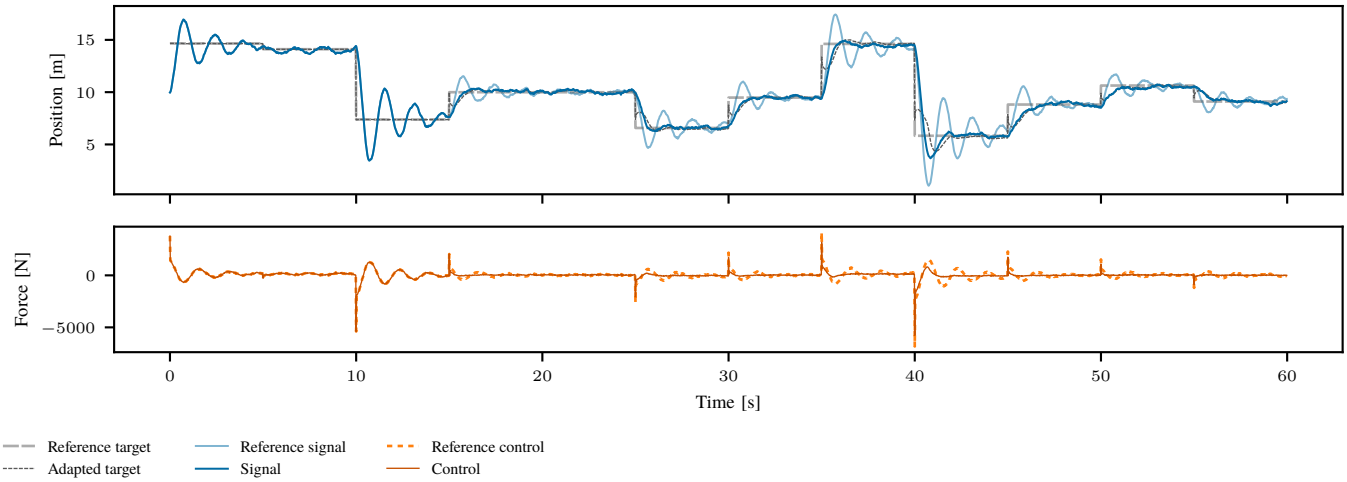


Fig. 4: The full minute simulation of the toy problem with an aggressive controller. Top: The system response x and target $x_d + \Delta$ of the adapted controller, alongside the response and target of the unadapted reference controller ($\Delta = 0$). Bottom: The resulting adapted control inputs from the controller receiving $x_d + \Delta$ alongside the unaltered reference control inputs. During the first 15 seconds (3 targets) the adapter has not received updates yet, and the signal is thus identical to the reference signal.

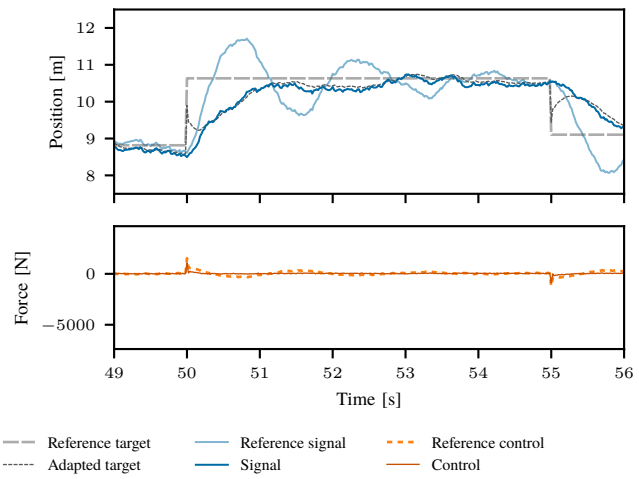


Fig. 5: A close-up of the second to last target in the one-minute simulation. The effects of the adapter can be more clearly observed: The overshoot (seen in the reference signal) is countered by bringing the target closer to the current state.

where the adapter would often encounter training features for which the labeled target position[1] x_d is somewhere in between x_t and $x_{t+window}$. Therefore, upon receiving the input feature $[\mathbf{x}_t, \mathbf{x}_d]$ it produces and adapted target position in between x_t and x_d. As the state approaches the adapted target, the adapter starts producing targets closer to the reference target, guiding the system towards this desired state.

2) A case where the controller is under-responsive: For a second simulation, the system and controller were designed such that the controller was more *conservative* or *under-responsive*. Additionally, the signal produced by the reference controller has a clear steady-state error. The result of the simulation can be found in fig. 6.

Once more, the adapter is able to improve the controller's performance after just a single update. In fig. 7 it can be seen that the adapted target is adjusted away from the current state. This is because training features created from the data in the buffer frequently include state pairs $\mathbf{x}_t$, $\mathbf{x}_{t+window}$ where the target position x_d is positioned further from x_t than it is from $x_{t+window}$. As a result, the adapter learns to predict adaptations that would move x_d farther from x_t, making the controller more aggressive. Another effect of the adapter is the consistent offset applied to the target: In the training features, the adapter regularly encounters pairs $\mathbf{x}_t, \mathbf{x}_{t+window} = \mathbf{x}_t$, which appear to represent the target state being reached, even though $x_{t+window} \neq x_d$. These features are the result of the steady-state error present in the signal, leading the adapter to learn to apply an offset to the target to counteract this.

IV. A Kresling Origami Robot

A. Mechanical Details

As mentioned in section II, origami robots—and by extension, soft robots—are a suitable use-case for lifelong learning techniques. To evaluate the method described in section III, a robot was designed that serves as a good analogue to the toy problem, whilst also using kresling origami as a key part of its functionality. Figure 8 provides important nomenclature for reference.

discuss this in section II

The robot is a simple crawler consisting of two Kresling origami springs (KOS) (fig. 8a, item 1) that are mirror images of each other, connected in series, with the actuation mechanism in between. Both springs can expand and contract in sync, producing a crawling-like motion when combined with

[1]Rather, the labeled $\Delta = x_d - x_{t+window}$, but the explanation provided is equivalent and somewhat clearer.

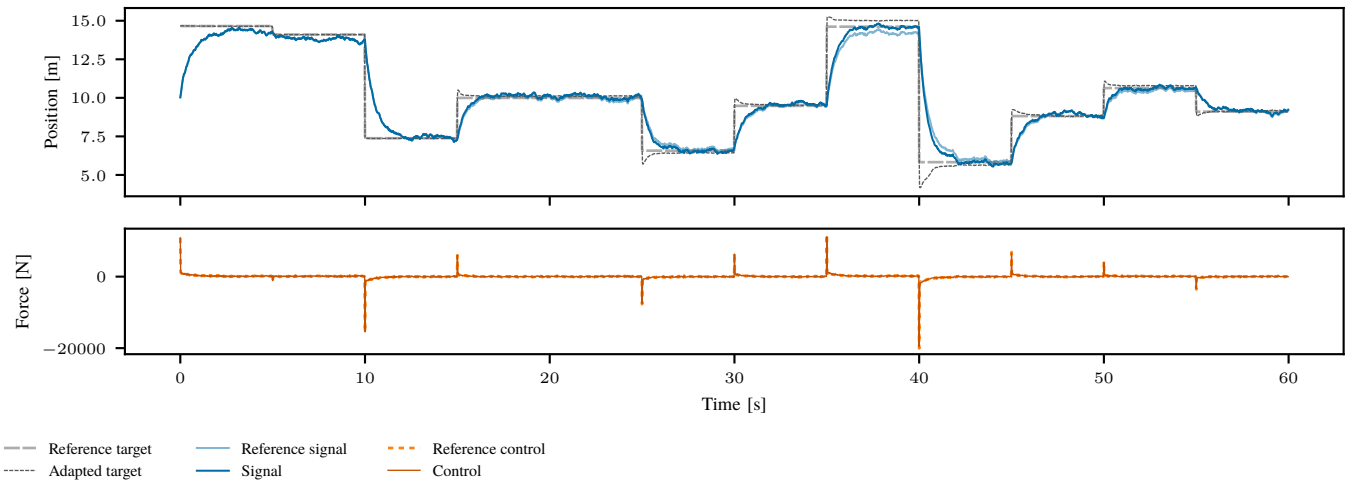


Fig. 6: The full minute simulation of the system with an under-responsive controller. Top: The system response and target of the adapted controller, alongside the response and target of the unadapted reference controller. Bottom: The resulting adapted control inputs from the controller receiving the altered target alongside the unaltered reference control inputs.

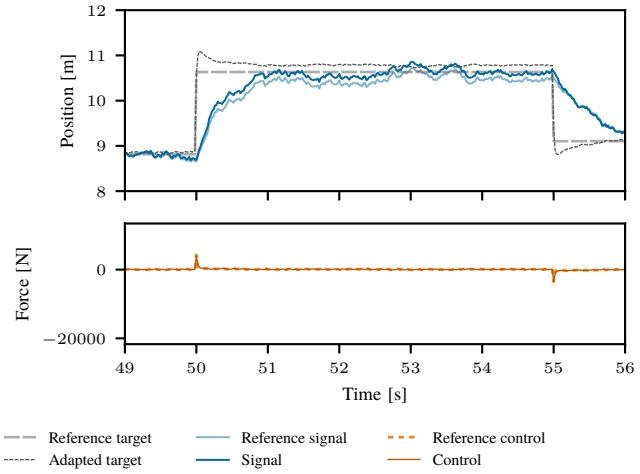


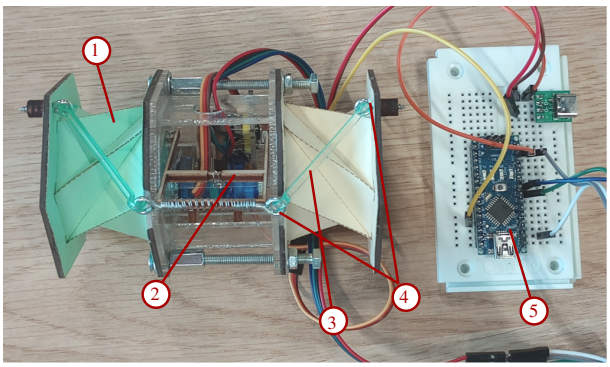
Fig. 7: A close-up of the second to last target in the one-minute simulation. Noteable observations: The target is initially moved further from the current state to 'pull' it towards the desired state in a shorter period. In addition, the adapted target is offset from the esired state as to counter the steady-state error seen in the reference signal.

specially designed "feet". To demonstrate target adaptation, however, the movement of the robot as a whole is not very important. Rather, the state of the KOSs is the state that will be controlled.

A KOS can be actuated both by linear motion (pushing or pulling on the spring) and by rotational motion (twisting). Since the two motions are directly related (section II-B), a contracting or expanding motion can be measured as the angle of the KOS' twist. The twist is induced by turning a rotating arm (2) which is connected to the ends of the KOSs by two links (3) that hook into eye-screws (4). To turn the arm, a micro servo motor (6) is employed. In the toy problem, the controller yields a force to exert on the system in order to reach the target position. However, most micro servo's come with a near perfect position controller built-in, which cannot be accessed or changed. Therefore, to bridge the gap between the simulation and the real-world implementation, the position output of the servo is converted to a force output by the use of a small, metal spring (8). The servo thus does not directly act on the actuation arm, but rotates a servo arm (7) instead, which pulls the spring and thereby exerts a force on the actuation arm. As previously mentioned, the state of the KOS can be measured as the angle of twist, since it is directly related to its contraction. Likewise, assuming perfectly rigid links with negligible clearance between the hooks and eye-screws, the angle of twist is directly related to the angle of the actuation arm. In the full implementation, the robot's state $\mathbf{x}_t$ will thus be measured as the actuation arm's angular position and velocity $[\theta, \dot{\theta}]$. This angle is measured using a contactless sensor (9) that can measure the on-axis rotation of a small magnet (10) attached to the actuation arm. The sensor relays this information to an Arduino Nano (5) board, which runs a PID controller that returns the control action, i.e., an angular position for the servo. The sensor measurement denotes a clockwise rotation (as viewed from the top) w.r.t. the datum as a positive angle, and likewise, a counterclockwise rotation as a decrement in the angle. In practice, this means that a positive control input from the servo (i.e. a positive angle) results in a decrement in θ. The operational range of the actuation arm is thus fully in the negative domain (fig. 9). Furthermore, there are mechanical limits for the rotation of both the actuation arm and servo arm, defined by the edge of the container housing the actuation mechanism and the supports holding the servo motor, respectively. However, the design is such that the servo's mechanical limit corresponds

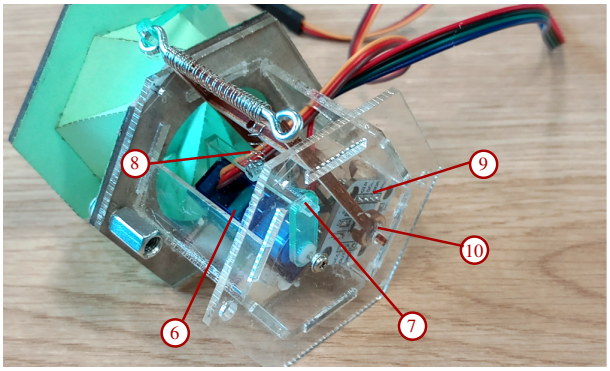
section II-B

cite ardui library



1) KOS
2) Actuation arm
3) Connecting link
4) Eye screw
5) Arduino

(a) The Kresling origami robot.



6) Servo motor
7) Servo arm
8) Metal spring
9) Angle sensor
10) Magnet

(b) Actuation mechanism.

Fig. 8: Annotated views of the robot and its most important actuation mechanisms

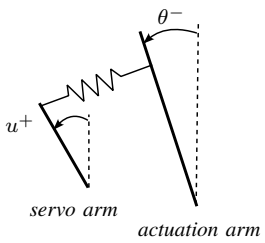


Fig. 9: Nominal signs of the angles for the servo and actuation arm during operation.

to a θ below the actuation arm's limit.

B. Software Implementation

The supervised learning implementation for target adaptations is divided into two key components: the nominal control of the robot, programmed in C++ and executed by the Arduino board, and the target adaptation, implemented in Python and managed by an external computer.

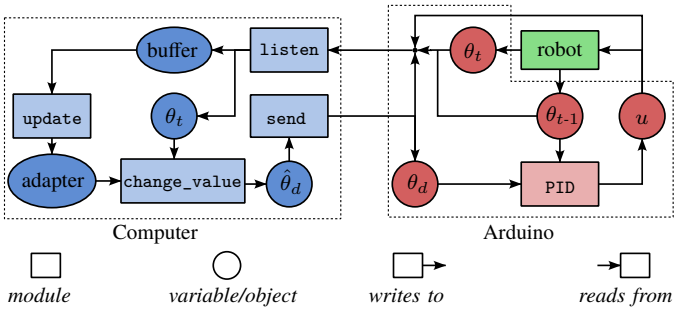


Fig. 10: The interaction between the modules and shared variables of the computer (blue) and Arduino (red).

1) Nominal Control on the Arduino: Upon startup, the Arduino initializes a connection with the external computer and directs the servo to move to its datum position. The Arduino allows some time for the servo to reach its datum and then reads a first measurement of the angle sensor, setting this as the reference $\theta = 0$. During the control loop, the servo reads θ (relative to the reference) and $\dot{\theta}$ received from the sensor, and reads the target θ_d received from the external computer. Based on these measurements, the PID controller computes the value to be written to the servo, but before writing it to the servo, a ceiling is imposed on it to protect the servo from exceeding its mechanical limit (section IV-A). The Arduino allows some time for the servo to act and then reads the measurements from the angle sensor again. Finally, it sends the tuple containing the state, action, resulting state, and target to the external computer.

2) Target Adaptation on the External Computer: The target adaptation and supervised learning are implemented exclusively on the external computer through the established connection with the Arduino. The program is divided into several modules or *threads* that run simultaneously. This way, values that need to be sent or read can be updated in the background upon request, without delaying the operations of other modules (see fig. 10 for reference).

possibly move this picture (a subsection to an appendix

For example, the sole task of the `send` thread is to read the value of a global variable representing the target state and send this to the Arduino. The `listen` thread reads the tuple sent in return by the Arduino and assigns its contents to the respective shared variables for the state, control action, resulting state and target. It then adds this tuple to the buffer used for updating the adapter model. The `update` thread is responsible for (re)training the adapter. It creates a copy of the adapter, which can then be trained in the background while the original adapter remains functional. Once training is completed, the copied adapter's weights are assigned to the original adapter. The `change_value` thread is responsible for setting the *true* target in the control loop, as well as the actual adaptations to it. At regular predefined intervals, it changes the true target θ_d to some value. Every iteration, it constructs an input feature $[\theta, \dot{\theta}, \theta_d, 0]$ for the adapter. The resulting predicted Δ is added to the target, which is read by the `send` thread and forwarded to the Arduino.